# Prompt-to-Scene Engine

*Technical Report — Condensed*

AI-Driven 2D Level Generation via NLP & Conditional Transformer Architecture

## 1. Project Overview

The Prompt-to-Scene Engine is an end-to-end AI pipeline that translates natural language descriptions into structurally valid, playable 2D game level blueprints. The system eliminates the manual bottleneck of level design by composing two specialized AI models in sequence:

- Model A – Semantic Entity Extractor: A fine-tuned DistilBERT NER model that parses free-form English and extracts game objects, quantities, positions, and scene themes.
- Model B – Conditional Architect Transformer: A custom autoregressive transformer trained on Super Mario Bros. level data that generates 16×16 tile layout blueprints conditioned on Model A's output.

## 2. Full Pipeline

The pipeline converts a raw text prompt into a rendered game scene through five sequential stages:

| Stage | Description | Technology |
|---|---|---|
| 1 | Semantic Entity Extraction — Model A (DistilBERT NER, fine-tuned on 5,000+ synthetic examples) | *Python / HuggingFace* |
| 2 | Post-Processing Engine — typo correction, plural normalization, hyphen-glue for Unity prefab names | *Python (spellchecker)* |
| 3 | Spatial Data Engineering — VGLC Mario dataset → Universal Token Set → 16×16 chunks | *NumPy / PyTorch* |
| 4 | Conditional Layout Generation — Model B (Custom Autoregressive Transformer with 2D positional encoding) | *PyTorch (CUDA)* |
| 5 | Blueprint-to-Scene Translation — JSON blueprint mapped to *SpriteStack* prefab components | *SpriteStack Engine / C#* |

### 2.1 Data Flow Summary

A user prompt such as "In a lava cave, spawn 3 fire dragons at the center and 2 skeletons on the left" flows through the system as follows:

- Model A extracts structured entities: { object: "giant-red-fire-dragon", count: 3, position: "center", scene_type: "lava-cave" }

- The post-processing engine corrects typos, normalizes plurals, and formats object names for Unity prefabs.
- A 27-dimensional condition vector encodes the spatial zone distribution of each entity class across a 3×3 grid overlaid on the level.
- Model B autoregressively generates a 16×16 tile matrix (Air, Solid, Loot, Enemy, Climbable, Player tokens).
- The Blueprint-to-Scene Translator maps each token to a *SpriteStack* prefab and renders the final game scene.

# 3. Model A — Semantic Entity Extractor

## 3.1 Architecture: DistilBERT

Model A is built on DistilBERT (Distilled BERT), a compact transformer produced via knowledge distillation from BERT-base. It retains ~95% of BERT's linguistic understanding at 40% fewer parameters and 60% faster inference. A token classification head is attached and fine-tuned for game-domain Named Entity Recognition (NER) using the HuggingFace Trainer API.

## 3.2 Synthetic Dataset

Since no annotated game-command NER dataset exists publicly, a Synthetic Dataset Generator was built in Python. It produced 5,000+ diverse labeled examples from a curated ontology:

- 50+ game objects (enemies, loot, furniture, environmental props)
- 10+ scene themes (lava-cave, dungeon, forest, castle, ice-tundra, etc.)
- Spatial positions: left, center, right, top, bottom, background, foreground
- Count patterns: numeric words ("three") and digits ("3")

Controlled variability was introduced via synonym substitution, sentence reordering, and typo injection to ensure generalization to real human input.

## 3.3 IOB Tagging & Training Results

The dataset was annotated using the IOB (Inside-Outside-Beginning) tagging scheme with labels: B-COUNT, I-COUNT, B-OBJECT, I-OBJECT, B-POSITION, B-SCENE_TYPE, and O. Model A was trained for 5 epochs:

| Epoch | Train Loss | Val Loss | Precision | Recall | F1 / Acc. |
|---|---|---|---|---|---|
| 1 | 0.003726 | 0.001962 | 1.000 | 1.000 | 1.000 |
| 2 | 0.001797 | 0.000970 | 1.000 | 1.000 | 1.000 |
| 3 | 0.001346 | 0.000696 | 1.000 | 1.000 | 1.000 |
| 4 | 0.001121 | 0.000590 | 1.000 | 1.000 | 1.000 |
| 5 | 0.001061 | 0.000560 | 1.000 | 1.000 | 1.000 |

Training and validation loss decreased steadily across all epochs (0.003726 → 0.001061 train; 0.001962 → 0.000560 val), confirming stable convergence with no divergence or overfitting.

### 3.4 Post-Processing Engine

After raw IOB tag prediction, a Python inference engine performs:

- Spell correction: 'buttom' → 'bottom', 'skeletton' → 'skeleton'
- Hyphen-glue: 'fire' + 'dragon' → 'fire-dragon' (Unity prefab naming)
- Theme inheritance: scene type propagated to all entity records in output JSON
- Plural normalization: 'goblins' → 'goblin'
- Count resolution: 'three' → 3; defaults to 1 if unspecified

## 4. Data Engineering for Model B

### 4.1 Dataset: Video Game Level Corpus (VGLC)

The Super Mario Bros. subset of the publicly available Video Game Level Corpus (VGLC) was used as primary training data. It provides ASCII-encoded levels with rich structural diversity: platforms, gaps, pipes, enemies, and rewards — matching the 2D platformer physics domain Model B must replicate.

### 4.2 Universal Token Set (UTS)

Heterogeneous ASCII legends across VGLC levels were unified into a single 6-token Universal Token Set:

| Token ID | Category | VGLC Mapping |
|---|---|---|
| 0 | Air | '-', ' ' → empty/sky tiles |
| 1 | Solid | 'X', 'S', 'B' → ground, platforms, pipes |
| 2 | Loot | '?', 'Q' → reward blocks, coins |
| 3 | Enemy | 'E', 'g', 'k' → Goombas, Koopas, NPCs |
| 4 | Climbable | '#' → vines, ladders |
| 5 | Player | 'P' → player spawn point |

### 4.3 Spatial Extraction & Tensor Compilation

Two normalization steps were applied to raw levels:

- Vertical padding: all levels padded to 16 tiles in height by filling the sky region with Air tokens.
- Sliding window extraction: 16×16 windows traversed each level horizontally with stride=8 (50% overlap), capturing thousands of spatial chunks while preserving context across boundaries.

All validated chunks were compiled into mario_training_data.npy — yielding 1,269 strictly formatted int64 tensors of shape (16, 16). Each tensor was validated for: solid floor presence, valid token range, and correct shape.

### 4.4 Dataset Enrichment & Player Spawn Integration

The original VGLC chunks lacked player spawn tokens (Token 5), as spawn coordinates are stored in separate game metadata. A data enrichment pipeline (enrich_dataset.py) resolved this:

- Each original chunk was retained and augmented with 3 spatial variants (player_left, player_center, player_right).
- Player tokens were placed only on valid ground coordinates: an air tile directly above a solid tile.
- The conditioning vector was expanded from 18 to 27 dimensions to include player spatial zone counts across a 3×3 grid.

Final outputs: enriched_mario_data.npy, enriched_mario_labels.json, and enriched_conditional_data.pt — consumed directly by the Model B training script.

## 5. Model B — Conditional Architect Transformer

### 5.1 Architecture

The ConditionalArchitectTransformer generates 16×16 tile layouts one token at a time in raster scan order (left-to-right, top-to-bottom). Key components:

- Token Embedding Layer: maps each of the 6 tile tokens to a dense embedding vector.
- 2D Positional Encoding: custom learnable row and column embeddings summed per tile position, preserving 2D spatial structure lost by naive sequence flattening.
- Transformer Decoder Blocks: multi-head self-attention with causal masking, followed by feed-forward layers and layer normalization.
- Condition Injection: the 27-dimensional condition vector is projected and added to token embeddings at every generation step.
- Output Projection: a linear layer maps hidden states to logits over the 6-token vocabulary; greedy decoding selects the highest-logit token at inference.

### 5.2 27-Dimensional Condition Vector

Each training chunk is paired with a condition vector computed by auto-annotation: a 3×3 spatial grid is overlaid on the 16×16 chunk and the proportion of tiles per entity class per zone is computed. The 27 values (3 categories × 9 zones) encode spatial intent:

- Dimensions [0–8]: Enemy tile counts per spatial zone.
- Dimensions [9–17]: Loot tile counts per spatial zone.
- Dimensions [18–26]: Player token counts per spatial zone.

At inference time, this vector is derived directly from Model A's output JSON, mapping entity positions and counts to the same 27-dimensional format.

### 5.3 Training Results

Model B was trained with GPU acceleration (CUDA) using cross-entropy loss over all 256 tile positions of the 16×16 grid. The model achieved a final training loss of ~0.0699, with smooth convergence across epochs and no divergence — confirming stable gradient flow through the 2D positional encoding and condition injection mechanism.

## 6. End-to-End Pipeline Integration

The end_to_end_test.py script connects both models into a single inference pass:

- 1. Raw text prompt → Model A tokenizes, predicts IOB tags, applies post-processing → structured JSON blueprint.
- 2. JSON entity positions → auto-annotation logic → 27-dimensional condition vector.
- 3. Condition vector → Model B autoregressively generates 256 tile tokens → 16×16 tile matrix.
- 4. Tile matrix validated (solid floor check, token range) → output as NumPy array / JSON blueprint.
- 5. Blueprint-to-Scene Translator maps tokens to *SpriteStack* prefabs and renders the final game scene.

The script achieves fully automated Prompt-in → Level-out generation with no manual intervention at any stage.